

Introduction to Database

Event Management System



American International University – Bangladesh

Spring 2023 – 2024

Course Name: Introduction to Database

Section: K

Group: 09

Faculty: Shahnaz Parvin

Member Name	ID
Samiha Chowdhury	23-50466-1
Tasfiah Tasnim Mrinmoyee	23-50399-1
Khan Abrar Muhtadi	21-44469-1
Mostafizur Rahman	23-50829-1

✂Context:

- Introduction [03]
- Project Description [03]
- ER Diagram [04]
- Normalization [04~07]
- Table Creation and Data Insertion [08~13]
- Query writing [14~20]
- Relational Algebra [21]
- Conclusion [22]

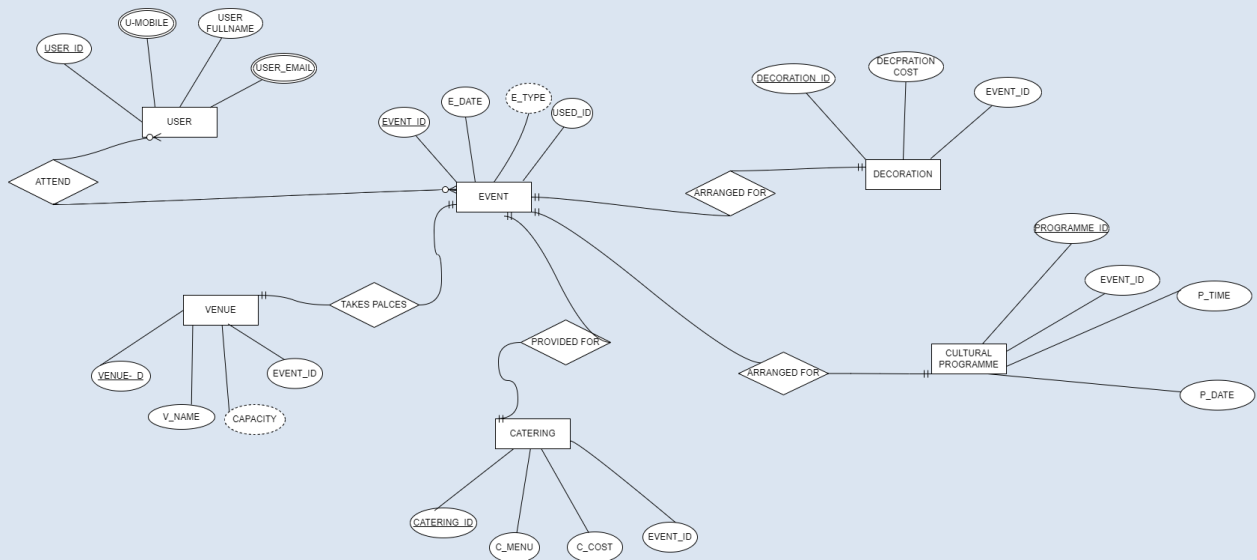
✧ **Project Title:** Event Management using Database System

- **Introduction:** Event management is the application of project management to the creation and development of small and/or large scale personal or corporate events such as, festivals, conferences, ceremonies, weddings, formal parties, concerts, or conventions. An event management system in a database is designed to handle all the aspects of organizing and running events.

StudentID1: 23- 50466-1 Name: Samiha Chowdhury	StudentID3: 21-44469-1 Name: Khan Abrar Muhtadi
StudentID2: 23-50399-1 Name: Tasfiah Tasnim Mrinmoyee	StudentID4: 23-50829-1 Name: Mostafizur Rahman
CO2: Understand the fundamental concepts underlying database systems and gain hands-on experience with ER diagram Case study	
PO-c2: Develop process for complex computer science and engineering problems considering cultural and societal factors.	Marks

Project Description: In an event management system, every Event is identified with event ID where event's name, date, and type are stored, additionally including user ID. An event is attended by user whose ID, name, mobile number, and email are stored and catering is provided here where the ID, menu, cost of the foods are stored. Again, the system is mentioned about venue that takes place in the event and its ID, name, Capacity are stored. Additionally, decoration is arranged in the Event where its ID and cost are stored. Lastly, the cultural programme which is arranged in the event stores its ID, timing and date, In the system, Venue, Catering, Decoration and Cultural Programme store event's ID in each of them.

➤ ER Diagram:



➤ Normalization:

NORMALIZATION:

ATTEND

ATTRIBUTES: USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL, EVENT_ID, EVENT_DATE, EVENT_TYPE

1NF: U_MOBILE AND USER_EMAIL ARE MULTIPLE ATTRIBUTE

1. USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL, EVENT_ID, EVENT_DATE, EVENT_TYPE

2NF:

1. USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL

2. EVENT_ID, EVENT_DATE, EVENT_TYPE

3NF: THERE IS NO TRANSITIVE DEPENDENCY

1. USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL

2. EVENT_ID, EVENT_DATE, EVENT_TYPE

TABLE CREATION:

1.USER- USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL

2.EVENT- EVENT_ID, EVENT_DATE, EVENT_TYPE, **USER_ID**

TAKES PLACES

ATTRIBUTES: EVENT_ID, EVENT_DATE, EVENT_TYPE, VENUE_ID, V_NAME, CAPACITY

1NF: THERE ARE NO MULTIPLE VALUE ATTRIBUTE.RELATION IS ALREADY IN 1NF

1. EVENT_ID, EVENT_DATE, EVENT_TYPE, VENUE_ID, V_NAME, CAPACITY

2NF:

1. EVENT_ID, EVENT_DATE, EVENT_TYPE,

2. VENUE_ID, V_NAME, CAPACITY

3NF: 1. EVENT_ID, EVENT_DATE, EVENT_TYPE,

2. VENUE_ID, V_NAME, CAPACITY

3. V_NAME, CAPACITY

TABLE CREATION:

1.EVENT- EVENT_ID, EVENT_DATE, EVENT_TYPE

2.VENUE-. VENUE_ID, V_NAME, CAPACITY, **EVENT_ID**

3.CAPACITY- V_NAME, CAPACITY, **V_NAME**

PROVIDED FOR

ATTRIBUTES: - EVENT_ID, EVENT_DATE, EVENT_TYPE, CATERING-ID, C-MENU, C_COST

1NF: THERE ARE NO MULTIPLE VALUE ATTRIBUTE.RELATION IS ALREADY IN 1NF

THERE ARE NO MULTIPLE VALUE ATTRIBUTE.RELATION IS ALREADY IN 1NF

2NF:

1. EVENT_ID, EVENT_DATE, EVENT_TYPE,

2. CATERING-ID, C-MENU, C_COST

3NF:

1. EVENT_ID, EVENT_DATE, EVENT_TYPE,
2. CATERING-ID, C-MENU, C_COST
3. CM-ID, C-MENU, C_COST

TABLE CREATION:

1. EVENT- EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**
2. CATERING- CATERING_ID, C-MENU, C_COST, **C_MENU**
3. C_MENU-CM-ID, C-MENU, C_COST

ARRANGED FOR

ATTRIBUTES: EVENT_ID, EVENT_DATE, EVENT_TYPE,
DECORATION_ID, DECORATION COST

1NF: THERE IS NO MULTIPLE VALUE ATTRIBUTE. RELATION IS
ALREADY IN 1NF.

1. EVENT_ID, EVENT_DATE, EVENT_TYPE, DECORATION_ID,
DECORATION COST

2NF:

1. EVENT_ID, EVENT_DATE, EVENT_TYPE
 2. DECORATION_ID, DECORATION COST
- 3NF:** THERE IS NO TRANSITIVE DEPENDENCY
1. EVENT_ID, EVENT_DATE, EVENT_TYPE
 2. DECORATION_ID, DECORATION COST

TABLE CREATION:

1. EVENT- EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**
2. DECORATION- DECORATION_ID, DECORATION COST

ARRANGED FOR

ATTRIBUTES: EVENT_ID, EVENT_DATE, EVENT_TYPE, PROGRAMME-
ID, PROGRAMME_TIME, P-DATE

1NF: THERE IS NO MULTIPLE VALUE ATTRIBUTE. RELATION IS
ALREADY IN 1NF

1. EVENT_ID, EVENT_DATE, EVENT_TYPE, PROGRAMME-ID,
PROGRAMME_TIME, P-DATE

2NF: .1. EVENT_ID, EVENT_DATE, EVENT_TYPE

2.PROGRAMME-ID, PROGRAMME_TIME, P-DATE

3NF:

1. EVENT_ID, EVENT_DATE, EVENT_TYPE

2.PROGRAMME-ID, PROGRAMME_TIME, P-DATE

3.PROGRMME-ID, P_DATE

TABLE CREATION:

1.EVENT- EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**

2.PROGRAMME- PROGRAMME-ID, PROGRAMME_TIME, **P-DATE**

3.P_ID- P_CODE, PROGRMME-ID, P_DATE

FINALIZATION:

TEMP TABLE:

1.USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL

2.EVENT_ID, EVENT_DATE, EVENT_TYPE, **USER_ID**

3.EVENT_ID, EVENT_DATE, EVENT_TYPE

4. VENUE_ID, V_NAME, CAPACITY, **EVENT_ID**

5.V_NAME, CAPACITY, **V_NAME**

6.EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**

7.CATERING_ID, C-MENU, C_COST, **C_MENU**

8.CM-ID, C-MENU, C_COST

9. EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**

10. DECORATION_ID, DECORATION COST

11. EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**

12. PROGRAMME-ID, PROGRAMME_TIME, **P-DATE**

13. P_CODE, PROGRMME-ID, P_DATE

FINAL TABLE:

1. USER_ID, U_MOBILE, USER_FULLNAME, USER_EMAIL

2. EVENT_ID, EVENT_DATE, EVENT_TYPE, **EVENT_ID**

3. V_NAME, CAPACITY, **V_NAME**

4. CATERING_ID, C-MENU, C_COST, **C_MENU**

5. CM-ID, C-MENU, C_COST

6. DECORATION_ID, DECORATION COST

7. PROGRAMME-ID, PROGRAMME_TIME, **P-DATE**

8. P_CODE, PROGRMME-ID, P_DATE

StudentID1: 23- 50466-1 Name: Samiha Chowdhury	StudentID3: 21-44469-1 Name: Khan Abrar Muhtadi
StudentID2: 23-50399-1 Name: Tasfiah Tasnim Mrinmoyee	StudentID4: 23-50829-1 Name: Mostafizur Rahman
CO4: Creating DML, DDL using Oracle and connection with ODBC/JDBC for existing JAVA application	
PO-e-2: Use modern engineering and IT tools for prediction and modeling of complex computer science and engineering problem	Marks

Table Creation and Data Insertion:

```
- create table event (
    e_id number(10) not null,
    e_type varchar(50) not null,
    e_date varchar(50) not null,
    user_id number(10) not null,
    primary key (e_id)
);
```

```
insert into event(e_id,e_type,e_date,user_id) values (01,'Marriage','22 Oct 2022',111);
insert into event(e_id,e_type,e_date,user_id) values (02,'conferences','24 Oct 2023',222);
insert into event(e_id,e_type,e_date,user_id) values (03,'Festivals','26 Oct 2023',333);
insert into event(e_id,e_type,e_date,user_id) values (04,'Networking ','28 Oct 2023',444);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
create table users (  
  user_id number(10) not null,  
  user_mob number(11) not null,  
  user_name varchar(50) not null,  
  user_email varchar(50) not null,  
  primary key (user_id)  
);  
  
insert into users(user_id,user_mob,user_name,user_email) values (111,01789564356,'Rifat','rifat@gmail.com');  
insert into users(user_id,user_mob,user_name,user_email) values (222,01789564357,'abir','abir@gmail.com');  
insert into users(user_id,user_mob,user_name,user_email) values (333,01789564358,'eram','eram@gmail.com');  
insert into users(user_id,user_mob,user_name,user_email) values (444,01789564359,'azim','azim@gmail.com');  
  
alter table event add constraint fk_user_id Foreign Key (user_id) references users(user_id);
```

The Results window shows the following data:

USER_ID	USER_MOB	USER_NAME	USER_EMAIL
111	1789564356	Rifat	rifat@gmail.com
222	1789564357	abir	abir@gmail.com
333	1789564358	eram	eram@gmail.com
444	1789564359	azim	azim@gmail.com

4 rows returned in 0.06 seconds

- create table users (
 user_id number(10) not null,
 user_mob number(11) not null,
 user_name varchar(50) not null,
 user_email varchar(50) not null,
 primary key (user_id)
);

insert into users(user_id,user_mob,user_name,user_email) values
(111,01789564356,'Rifat','rifat@gmail.com');
insert into users(user_id,user_mob,user_name,user_email) values
(222,01789564357,'abir','abir@gmail.com');
insert into users(user_id,user_mob,user_name,user_email) values
(333,01789564358,'eram','eram@gmail.com');
insert into users(user_id,user_mob,user_name,user_email) values
(444,01789564359,'azim','azim@gmail.com');

alter table event add constraint fk_user_id Foreign Key (user_id)
references users(user_id);

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```

create table event (
  e_id number(10) not null,
  e_type varchar(50) not null,
  e_date varchar(50) not null,
  user_id number(10) not null,
  primary key (e_id),
  foreign key (user_id)
);

insert into event(e_id,e_type,e_date,user_id) values (01,'Marriage','22 Oct 2022',111);
insert into event(e_id,e_type,e_date,user_id) values (02,'conferences','24 Oct 2023',222);
insert into event(e_id,e_type,e_date,user_id) values (03,'Festivals','26 Oct 2023',333);
insert into event(e_id,e_type,e_date,user_id) values (04,'Networking ','28 Oct 2023',444);

drop table event;

```

The Results tab shows the following data:

E_ID	E_TYPE	E_DATE	USER_ID
1	Marriage	22 Oct 2022	111
2	conferences	24 Oct 2023	222
3	Festivals	26 Oct 2023	333
4	Networking	28 Oct 2023	444

4 rows returned in 0.02 seconds

- create table venue (
 - v_id number(10) not null,
 - v_name varchar(50) not null,
 - e_id number(10),
 - primary key (v_id),
 - FOREIGN KEY (e_id) REFERENCES event(e_id)
);

insert into venue(v_id,v_name,e_id) values (1,'Dhanmondi',01);
 insert into venue(v_id,v_name,e_id) values (2,'Mirpur', 02);
 insert into venue(v_id,v_name,e_id) values (3,'Kuril', 03);
 insert into venue(v_id,v_name,e_id) values (4,'Badda', 04);

The screenshot shows the Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL Commands window contains the following code:

```
create table venue (  
  v_id number(10) not null,  
  v_name varchar(50) not null,  
  e_id number(10),  
  primary key (v_id),  
  FOREIGN KEY (e_id) REFERENCES event(e_id)  
);  
  
insert into venue(v_id,v_name,e_id) values (1,'Dhanmondi',01);  
insert into venue(v_id,v_name,e_id) values (2,'Mirpur', 02);  
insert into venue(v_id,v_name,e_id) values (3,'Kuril', 03);  
insert into venue(v_id,v_name,e_id) values (4,'Badda', 04);
```

The Results window shows the following data:

V_ID	V_NAME	E_ID
1	Dhanmondi	1
2	Mirpur	2
3	Kuril	3
4	Badda	4

4 rows returned in 0.02 seconds

- create table decoration (
 d_id number(10) not null,
 d_cost number(10) not null,
 e_id number(10),
 primary key (d_id),
 foreign key (e_id) references event(e_id)
);

insert into decoration (d_id,d_cost,e_id) values (1,10000,01);
insert into decoration (d_id,d_cost,e_id) values (2,12000,02);
insert into decoration (d_id,d_cost,e_id) values (3,8000,03);
insert into decoration (d_id,d_cost,e_id) values (4,9000,04);

The screenshot shows the Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL Commands window contains the following SQL code:

```
create table decoration (  
  d_id number(10) not null,  
  d_cost number(10) not null,  
  e_id number(10),  
  primary key (d_id),  
  foreign key (e_id) references event(e_id)  
);  
  
insert into decoration (d_id,d_cost,e_id) values (1,10000,01);  
insert into decoration (d_id,d_cost,e_id) values (2,12000,02);  
insert into decoration (d_id,d_cost,e_id) values (3,8000,03);  
insert into decoration (d_id,d_cost,e_id) values (4,9000,04);
```

The Results window shows the following data:

D_ID	D_COST	E_ID
1	10000	1
2	12000	2
3	8000	3
4	9000	4

4 rows returned in 0.02 seconds

- create table programme (
 p_id number(10) not null,
 p_time varchar(50) not null,
 p_date varchar(50) not null,
 e_id number(10),
 primary key (p_id),
 foreign key (e_id) references event(e_id)
);

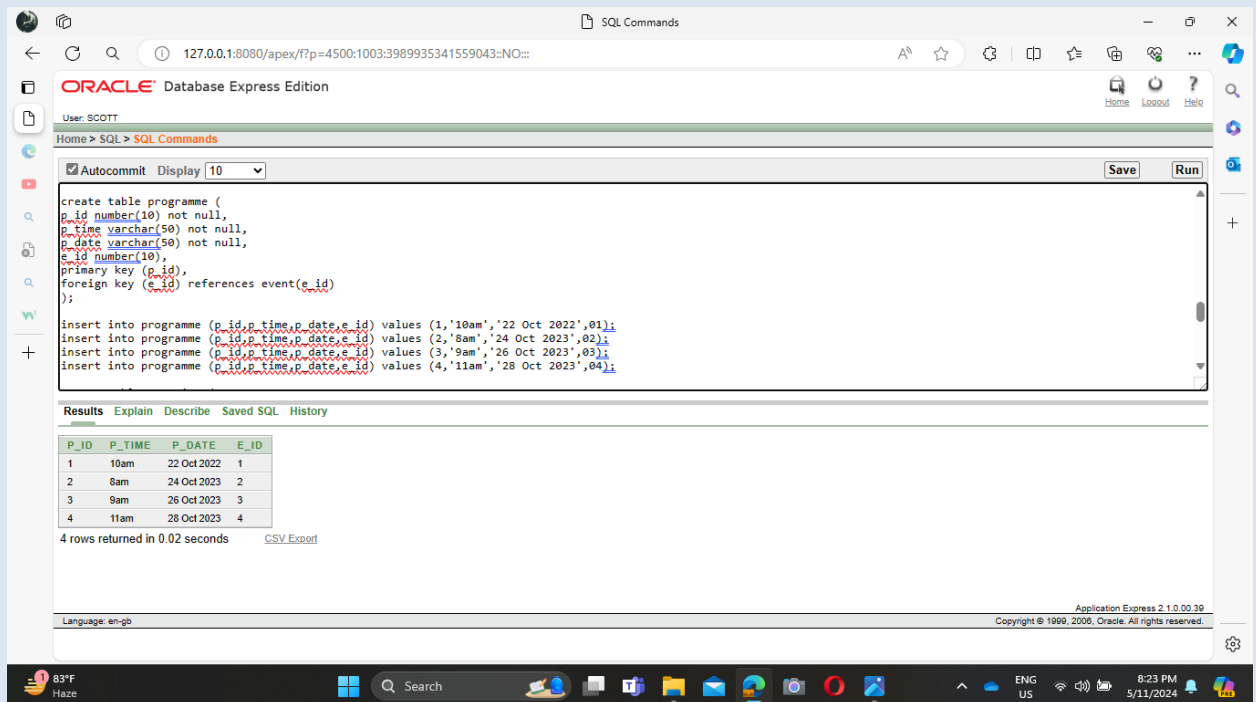
insert into programme (p_id,p_time,p_date,e_id) values
(1,'10am','22 Oct 2022',01);

insert into programme (p_id,p_time,p_date,e_id) values

```

(2,'8am','24 Oct 2023',02);
insert into programme (p_id,p_time,p_date,e_id) values
(3,'9am','26 Oct 2023',03);
insert into programme (p_id,p_time,p_date,e_id) values
(4,'11am','28 Oct 2023',04);

```



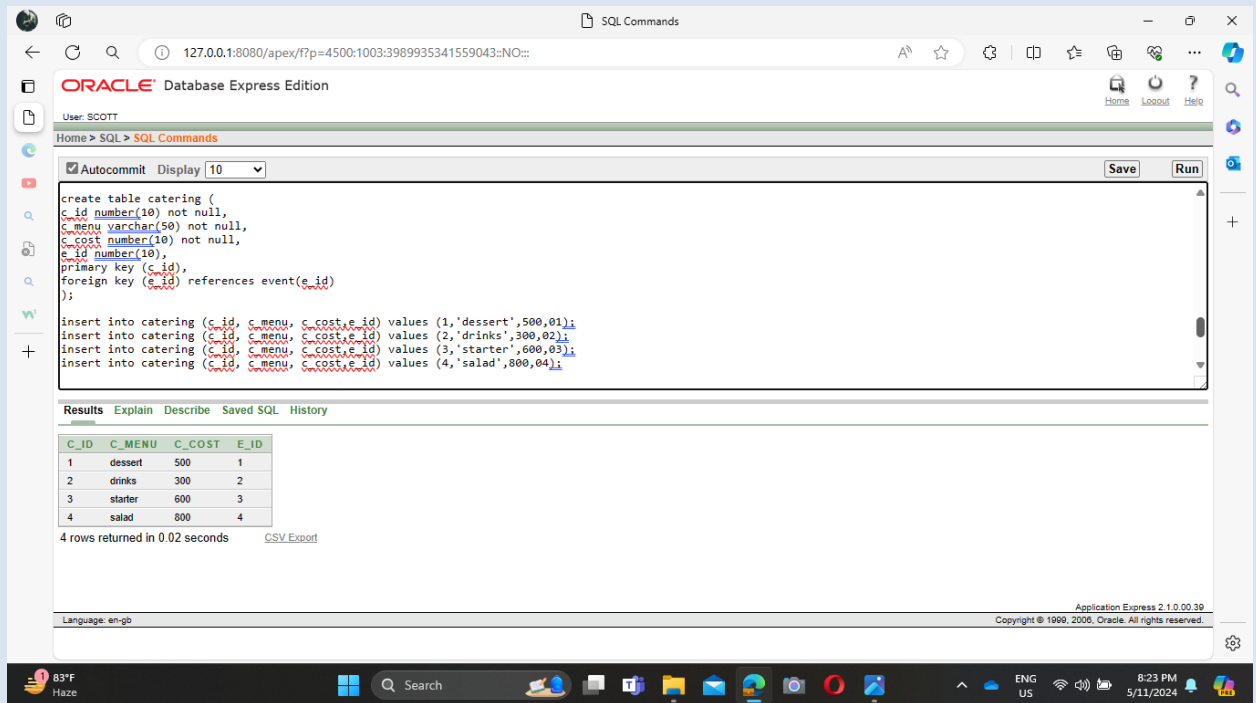
- create table catering (
 - c_id number(10) not null,
 - c_menu varchar(50) not null,
 - c_cost number(10) not null,
 - e_id number(10),
 - primary key (c_id),
 - foreign key (e_id) references event(e_id)
);

```

insert into catering (c_id, c_menu, c_cost,e_id) values
(1,'dessert',500,01);
insert into catering (c_id, c_menu, c_cost,e_id) values
(2,'drinks',300,02);

```

insert into catering (c_id, c_menu, c_cost,e_id) values
(3,'starter',600,03);
insert into catering (c_id, c_menu, c_cost,e_id) values
(4,'salad',800,04);



The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
create table catering (  
  c_id number(10) not null,  
  c_menu varchar(50) not null,  
  c_cost number(10) not null,  
  e_id number(10),  
  primary key (c_id),  
  foreign key (e_id) references event(e_id)  
);  
  
insert into catering (c_id, c_menu, c_cost,e_id) values (1,'dessert',500,01);  
insert into catering (c_id, c_menu, c_cost,e_id) values (2,'drinks',300,02);  
insert into catering (c_id, c_menu, c_cost,e_id) values (3,'starter',600,03);  
insert into catering (c_id, c_menu, c_cost,e_id) values (4,'salad',800,04);
```

The Results window shows the following data:

C_ID	C_MENU	C_COST	E_ID
1	dessert	500	1
2	drinks	300	2
3	starter	600	3
4	salad	800	4

4 rows returned in 0.02 seconds

➤ Query Writing:

___Simple Query___

1. Retrieve all events with their types and dates.

```
SELECT e_id, e_type, e_date
FROM event;
```

Results Explain Describe Saved S

E_ID	E_TYPE	E_DATE
1	Marriage	22 Oct 2022
2	conferences	24 Oct 2023
3	Festivals	26 Oct 2023
4	Networking	28 Oct 2023

4 rows returned in 0.00 seconds

Single Row Subqueries

2. Retrieve the user name for a specific event.

```
SELECT e_id, e_type, e_date,
       (SELECT user_name FROM users WHERE user_id = event.user_id) AS user_name
FROM event;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	USER_NAME
1	Marriage	22 Oct 2022	Rifat
2	conferences	24 Oct 2023	abir
3	Festivals	26 Oct 2023	eram
4	Networking	28 Oct 2023	azim

4 rows returned in 0.00 seconds

[CSV Export](#)

3. Calculate the total cost of decoration for an event.


```
SELECT e_id, e_type, e_date,
       (SELECT SUM(d_cost) FROM decoration WHERE e_id = event.e_id) AS total_decoration_cost
FROM event;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	TOTAL_DECORATION_COST
1	Marriage	22 Oct 2022	10000
2	conferences	24 Oct 2023	12000
3	Festivals	26 Oct 2023	8000
4	Networking	28 Oct 2023	9000

4 rows returned in 0.00 seconds [CSV Export](#)

____Multiple Row Subqueries____

4. Retrieve all events for a specific user.

```
SELECT user_id, user_name,
       (SELECT e_id FROM event WHERE user_id = users.user_id) AS event_id
FROM users;
```

Results Explain Describe Saved SQL History

USER_ID	USER_NAME	EVENT_ID
111	Rifat	1
222	abir	2
333	eram	3
444	azim	4

4 rows returned in 0.00 seconds [CSV Export](#)

5. Retrieve all catering menu options for an

event.

```
SELECT e_id, e_type, e_date,  
       (SELECT c_menu FROM catering WHERE e_id = event.e_id) AS catering_menu  
FROM event;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	CATERING_MENU
1	Marriage	22 Oct 2022	dessert
2	conferences	24 Oct 2023	drinks
3	Festivals	26 Oct 2023	starter
4	Networking	28 Oct 2023	salad

4 rows returned in 0.00 seconds

[CSV Export](#)

____Joins____

6. Use an inner join to retrieve events with users.

```
SELECT event.e_id, event.e_type, event.e_date, users.user_name  
FROM event  
INNER JOIN users ON event.user_id = users.user_id;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	USER_NAME
1	Marriage	22 Oct 2022	Rifat
2	conferences	24 Oct 2023	abir
3	Festivals	26 Oct 2023	eram
4	Networking	28 Oct 2023	azim

4 rows returned in 0.01 seconds

[CSV Export](#)

7. Utilize a left join to retrieve events with venue information.

```
SELECT event.e_id, event.e_type, event.e_date, venue.v_name
FROM event
LEFT JOIN venue ON event.e_id = venue.e_id;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	V_NAME
1	Marriage	22 Oct 2022	Dhanmondi
2	conferences	24 Oct 2023	Mirpur
3	Festivals	26 Oct 2023	Kuril
4	Networking	28 Oct 2023	Badda

4 rows returned in 0.00 seconds

[CSV Export](#)

8. Employ a right join to retrieve events with decoration information.

```
SELECT event.e_id, event.e_type, event.e_date, decoration.d_cost
FROM event
RIGHT JOIN decoration ON event.e_id = decoration.e_id;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	D_COST
1	Marriage	22 Oct 2022	10000
2	conferences	24 Oct 2023	12000
3	Festivals	26 Oct 2023	8000
4	Networking	28 Oct 2023	9000

4 rows returned in 0.00 seconds

[CSV Export](#)

9. Use a full outer join to retrieve events with programme information.

```
SELECT event.e_id, event.e_type, event.e_date, programme.p_time  
FROM event  
FULL OUTER JOIN programme ON event.e_id = programme.e_id;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	P_TIME
1	Marriage	22 Oct 2022	10am
2	conferences	24 Oct 2023	8am
3	Festivals	26 Oct 2023	9am
4	Networking	28 Oct 2023	11am

4 rows returned in 0.00 seconds

[CSV Export](#)

___Simple View___

10a. Create a simple view to display events and users.

```
CREATE VIEW event_user_view AS  
SELECT event.e_id, event.e_type, event.e_date, users.user_name  
FROM event  
INNER JOIN users ON event.user_id = users.user_id;
```

Results Explain Describe Saved SQL History

View created.

0.00 seconds

10b. Describe the simple view.

Object Type VIEW Object EVENT_USER_VIEW		Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
		EVENT_USER_VIEW	E_ID	Number	-	10	0	-	-	-	-
			E_TYPE	Varchar2	50	-	-	-	-	-	-
			E_DATE	Varchar2	50	-	-	-	-	-	-
			USER_NAME	Varchar2	50	-	-	-	-	-	-
		1 - 4									

10c. View the simple view as a complete table.

```
SELECT * FROM event_user_view;
```

Results Explain Describe Saved SQL History

E_ID	E_TYPE	E_DATE	USER_NAME
1	Marriage	22 Oct 2022	Rifat
2	conferences	24 Oct 2023	abir
3	Festivals	26 Oct 2023	eram
4	Networking	28 Oct 2023	azim

4 rows returned in 0.00 seconds

[CSV Export](#)

___Complex View___

12a. Create a complex view to display event details with total decoration cost.

```
CREATE VIEW event_decoration_cost_view AS
SELECT e_id, e_type, e_date,
       (SELECT SUM(d_cost) FROM decoration WHERE e_id = event.e_id) AS total_decoration_cost
FROM event;
```

Results Explain Describe Saved SQL History

View created.

0.02 seconds

12b. Describe the complex view.

Object Type **VIEW** Object **EVENT_DECORATION_COST_VIEW**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EVENT_DECORATION_COST_VIEW	E_ID	Number	-	10	0	-	-	-	-
	E_TYPE	Varchar2	50	-	-	-	-	-	-
	E_DATE	Varchar2	50	-	-	-	-	-	-
	TOTAL_DECORATION_COST	Number	-	-	-	-	✓	-	-
1 - 4									

12c. View the complex view as a complete table.

```
SELECT * FROM event_decoration_cost_view;
```

Results **Explain** **Describe** **Saved SQL** **History**

E_ID	E_TYPE	E_DATE	TOTAL_DECORATION_COST
1	Marriage	22 Oct 2022	10000
2	conferences	24 Oct 2023	12000
3	Festivals	26 Oct 2023	8000
4	Networking	28 Oct 2023	9000

4 rows returned in 0.02 seconds

[CSV Export](#)

➤ Relational Algebra

Question - Retrieve all events along with their types and dates.

Relational Algebra - $\pi_{e_id, e_type, e_date}(\text{Event})$

Description - This SQL query selects the "e_id", "e_type", and "e_date" columns from the "event" table.

Question - Retrieve user name for a specific event (e.g., event with e_id = 01).

Relational Algebra - $\pi_{user_name}(\text{Event} \bowtie_{\text{Event.user_id} = \text{Users.user_id}} \text{Users})$

Description - This SQL query retrieves the "user_name" for events with e_id = '01' by joining the "Event" and "Users" tables on the "user_id" attribute and selecting only the "user_name" attribute.

Question - Calculate total cost of decoration for an event (e.g., event with e_id = 01).

Relational Algebra - $\pi_{\text{SUM}(d_cost) \text{ AS total_decoration_cost}}(\text{Decoration} \mid e_id = '01')$

Description - This SQL query calculates the sum of the "d_cost" attribute from the "Decoration" table for the event with e_id = '01'.

Question - Retrieve all events for a specific user (e.g., user with user_id = 111).

Relational Algebra - $\sigma_{user_id = '111'}(\text{Event})$

Description - This SQL query retrieves all events where the user_id is '111'.

Question - Retrieve all catering menu options for an event (e.g., event with e_id = 01).

Relational Algebra - $\pi_{c_menu} \sigma_{e_id = '01'}(\text{Catering})$

Description - This SQL query retrieves the "c_menu" attribute from the "Catering" table for the event with e_id = '01'.

➤ Conclusion

Creating and running an event management system is considered complex. Understanding why it's important, where to start, and how to facilitate its growth are viewed as crucial elements for success.

The system is designed with the intention of simplifying event planning processes. Assistance is provided in tasks such as determining the event type, selecting dates and venues, and maintaining attendee records. Tools are offered to assist event organizers in communicating with attendees, disseminating updates, and collecting feedback. Support is also extended in the promotion of events to attract a larger audience.

Effective event management entails ensuring attendee satisfaction, delivering quality service, and increasing event visibility. The system is perceived as facilitating these objectives and more for event organizers. The overarching goal is to streamline event planning processes and enhance the enjoyment of events for all involved parties. With our assistance, organizers are enabled to orchestrate exceptional events that are highly anticipated and enjoyed by attendees.

The End

Thank You